

Module#

<https://pytorch.org/docs/stable/generated/torch.nn.Module.html>

Description:

Base class for all neural network modules. Your models should also subclass this class. Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes: Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Function Signature

```
class torch.nn.Module(*args, **kwargs)[source]#
```

Variables

```
add_module(name, module)[source]#
```

Parameters

```
apply(fn)[source]#
```

Returns:

Module

Code Examples

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

```

>>> @torch.no_grad()
>>> def init_weights(m):
>>>     print(m)
>>>     if type(m) == nn.Linear:
>>>         m.weight.fill_(1.0)
>>>         print(m.weight)
>>> net = nn.Sequential(nn.Linear(2, 2), nn.Linear(2, 2))
>>> net.apply(init_weights)
Linear(in_features=2, out_features=2, bias=True)
Parameter containing:
tensor([[1., 1.],
        [1., 1.]], requires_grad=True)
Linear(in_features=2, out_features=2, bias=True)
Parameter containing:
tensor([[1., 1.],
        [1., 1.]], requires_grad=True)
Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): Linear(in_features=2, out_features=2, bias=True)
)

```

```

>>> for buf in model.buffers():
>>>     print(type(buf), buf.size())
<class 'torch.Tensor'> (20L,)
<class 'torch.Tensor'> (20L, 1L, 5L, 5L)

```

```

A(
    (net_b): Module(
        (net_c): Module(
            (conv): Conv2d(16, 33, kernel_size=(3, 3), stride=
(2, 2))
        )
        (linear): Linear(in_features=100, out_features=200,
bias=True)
    )
)

```

```

>>> l = nn.Linear(2, 2)
>>> net = nn.Sequential(l, l)
>>> for idx, m in enumerate(net.modules()):
...     print(idx, '->', m)

0 -> Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): Linear(in_features=2, out_features=2, bias=True)
)
1 -> Linear(in_features=2, out_features=2, bias=True)

```

```

>>> for name, buf in self.named_buffers():
>>>     if name in ['running_var']:
>>>         print(buf.size())

```

```

>>> for name, module in model.named_children():
>>>     if name in ['conv4', 'conv5']:
>>>         print(module)

```

```
>>> l = nn.Linear(2, 2)
>>> net = nn.Sequential(l, l)
>>> for idx, m in enumerate(net.named_modules()):
...     print(idx, '->', m)

0 -> ('', Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): Linear(in_features=2, out_features=2, bias=True)
))
1 -> ('0', Linear(in_features=2, out_features=2, bias=True))
```

```
>>> for name, param in self.named_parameters():
>>>     if name in ['bias']:
>>>         print(param.size())
```

```
>>> for param in model.parameters():
>>>     print(type(param), param.size())
<class 'torch.Tensor'> (20L,)
<class 'torch.Tensor'> (20L, 1L, 5L, 5L)
```

```
>>> self.register_buffer('running_mean',
torch.zeros(num_features))
```

```
hook(module, args, output) -> None or modified output
```

Notes & Warnings

NOTE Note As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

NOTE Note This method modifies the module in-place.

NOTE Note This method modifies the module in-place.

NOTE Note This method modifies the module in-place.

NOTE Note This method modifies the module in-place.

NOTE Note This method modifies the module in-place.

NOTE Note Although the recipe for forward pass needs to be defined within this function, one should call the Module instance afterwards instead of this since the former takes care of running the registered hooks while the latter silently ignores them.

NOTE Note This method modifies the module in-place.

NOTE Note This method modifies the module in-place.

⚠ WARNING Warning If assign is True the optimizer must be created after the call to load_state_dict unless get_swap_module_params_on_conversion() is True.

NOTE Note If a parameter or buffer is registered as None and its corresponding key exists in state_dict, load_state_dict() will raise a RuntimeError.

NOTE Note Duplicate modules are returned only once. In the following example, l will be returned only once.

NOTE Note This method modifies the module in-place.

NOTE Note Duplicate modules are returned only once. In the following example, l will be returned only once.

 **WARNING** Warning Modifying inputs or outputs inplace is not allowed when using backward hooks and will raise an error.

 **WARNING** Warning Modifying inputs inplace is not allowed when using backward hooks and will raise an error.

 **NOTE** Note If strict is set to False (default), the method will replace an existing submodule or create a new submodule if the parent module exists. If strict is set to True, the method will only attempt to replace an existing submodule and throw an error if the submodule does not exist.

 **NOTE** Note The returned object is a shallow copy. It contains references to the module's parameters and buffers.

 **WARNING** Warning Currently `state_dict()` also accepts positional arguments for destination, prefix and keep_vars in order. However, this is being deprecated and keyword arguments will be enforced in future releases.

 **WARNING** Warning Please avoid the use of argument destination as it is not designed for end-users.

NOTE This method modifies the module in-place.

NOTE This method modifies the module in-place.

NOTE This method modifies the module in-place.

Detailed Documentation

Documentation

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

Add a child module to the current module.

The module can be accessed as an attribute using the given name.

name (str) – name of the child module. The child module can be accessed from this module using the given name

module (Module) – child module to be added to the module.

Apply fn recursively to every submodule (as returned by `.children()`) as well as self.

Typical use includes initializing the parameters of a model (see also `torch.nn.init`).

fn (Module -> None) – function to be applied to each submodule

Casts all floating point parameters and buffers to `bfloat16` datatype.

This method modifies the module in-place.

Return an iterator over module buffers.

recurse (bool) – if `True`, then yields buffers of this module and all submodules. Otherwise, yields only buffers that are direct members of this module.

`torch.Tensor` – module buffer

Return an iterator over immediate children modules.

Module – a child module

Compile this Module's forward using `torch.compile()`.

This Module's `__call__` method is compiled and all arguments are passed as-is to `torch.compile()`.

See `torch.compile()` for details on the arguments for this function.

Move all model parameters and buffers to the CPU.

This method modifies the module in-place.

Move all model parameters and buffers to the GPU.

This also makes associated parameters and buffers different objects. So it should be called before constructing the optimizer if the module will live on GPU while being optimized.

This method modifies the module in-place.

`device (int, optional)` – if specified, all parameters will be copied to that device

Casts all floating point parameters and buffers to double datatype.

This method modifies the module in-place.

Set the module in evaluation mode.

This has an effect only on certain modules. See the documentation of particular modules for details of their behaviors in training/evaluation mode, i.e. whether they are affected, e.g. Dropout, BatchNorm, etc.

This is equivalent with `self.train(False)`.

See [Locally disabling gradient computation](#) for a comparison between `.eval()` and several similar mechanisms that may be confused with it.

Return the extra representation of the module.

To print customized extra information, you should re-implement this method in your own modules. Both single-line and multi-line strings are acceptable.

Casts all floating point parameters and buffers to float datatype.

This method modifies the module in-place.

Define the computation performed at every call.

Should be overridden by all subclasses.

Although the recipe for forward pass needs to be defined within this function, one should call the Module instance afterwards instead of this since the former takes care of running the registered hooks while the latter silently ignores them.

Return the buffer given by target if it exists, otherwise throw an error.

See the docstring for `get_submodule` for a more detailed explanation of this method's functionality as well as how to correctly specify target.

`target (str)` – The fully-qualified string name of the buffer to look for. (See `get_submodule` for how to specify a fully-qualified string.)

The buffer referenced by target

`AttributeError` – If the target string references an invalid path or resolves to something that is not a buffer

Return any extra state to include in the module's `state_dict`.

Implement this and a corresponding `set_extra_state()` for your module if you need to store extra state. This function is called when building the module's `state_dict()`.

Note that extra state should be picklable to ensure working serialization of the `state_dict`. We only provide backwards compatibility guarantees for serializing Tensors; other objects may break backwards compatibility if their serialized pickled form changes.

Any extra state to store in the module's `state_dict`

Return the parameter given by `target` if it exists, otherwise throw an error.

See the docstring for `get_submodule` for a more detailed explanation of this method's functionality as well as how to correctly specify `target`.

`target` (str) – The fully-qualified string name of the `Parameter` to look for. (See `get_submodule` for how to specify a fully-qualified string.)

The `Parameter` referenced by `target`

`AttributeError` – If the `target` string references an invalid path or resolves to something that is not an `nn.Parameter`

Return the submodule given by `target` if it exists, otherwise throw an error.

For example, let's say you have an `nn.Module A` that looks like this:

(The diagram shows an `nn.Module A`. `A` which has a nested submodule `net_b`, which itself has two submodules `net_c` and `linear`. `net_c` then has a submodule `conv`.)

To check whether or not we have the `linear` submodule, we would call `get_submodule("net_b.linear")`. To check whether we have the `conv` submodule, we would call `get_submodule("net_b.net_c.conv")`.

The runtime of `get_submodule` is bounded by the degree of module nesting in `target`. A query against `named_modules` achieves the same result, but it is $O(N)$ in the number of

transitive modules. So, for a simple check to see if some submodule exists, `get_submodule` should always be used.

`target (str)` – The fully-qualified string name of the submodule to look for. (See above example for how to specify a fully-qualified string.)

The submodule referenced by `target`

`AttributeError` – If at any point along the path resulting from the `target` string the (sub)path resolves to a non-existent attribute name or an object that is not an instance of `nn.Module`.

Casts all floating point parameters and buffers to half datatype.

This method modifies the module in-place.

Move all model parameters and buffers to the IPU.

This also makes associated parameters and buffers different objects. So it should be called before constructing the optimizer if the module will live on IPU while being optimized.

This method modifies the module in-place.

`device (int, optional)` – if specified, all parameters will be copied to that device

Copy parameters and buffers from `state_dict` into this module and its descendants.

If `strict` is `True`, then the keys of `state_dict` must exactly match the keys returned by this module's `state_dict()` function.

If `assign` is `True` the optimizer must be created after the call to `load_state_dict` unless `get_swap_module_params_on_conversion()` is `True`.

`state_dict` (dict) – a dict containing parameters and persistent buffers.

`strict` (bool, optional) – whether to strictly enforce that the keys in `state_dict` match the keys returned by this module's `state_dict()` function. Default: True

`assign` (bool, optional) – When set to False, the properties of the tensors in the current module are preserved whereas setting it to True preserves properties of the Tensors in the state dict. The only exception is the `requires_grad` field of Parameter for which the value from the module is preserved. Default: False

by this module but missing from the provided `state_dict`.

expected by this module but present in the provided `state_dict`.

NamedTuple with `missing_keys` and `unexpected_keys` fields

If a parameter or buffer is registered as None and its corresponding key exists in `state_dict`, `load_state_dict()` will raise a `RuntimeError`.

Return an iterator over all modules in the network.

Module – a module in the network

Duplicate modules are returned only once. In the following example, I will be returned only once.

Move all model parameters and buffers to the MTIA.

This also makes associated parameters and buffers different objects. So it should be called before constructing the optimizer if the module will live on MTIA while being optimized.

This method modifies the module in-place.

device (int, optional) – if specified, all parameters will be copied to that device

Return an iterator over module buffers, yielding both the name of the buffer as well as the buffer itself.

prefix (str) – prefix to prepend to all buffer names.

recurse (bool, optional) – if True, then yields buffers of this module and all submodules. Otherwise, yields only buffers that are direct members of this module. Defaults to True.

remove_duplicate (bool, optional) – whether to remove the duplicated buffers in the result. Defaults to True.

(str, torch.Tensor) – Tuple containing the name and buffer

Iterator[tuple[str, torch.Tensor]]

Return an iterator over immediate children modules, yielding both the name of the module as well as the module itself.

(str, Module) – Tuple containing a name and child module

Iterator[tuple[str, Module]]

Return an iterator over all modules in the network, yielding both the name of the module as well as the module itself.

memo (Optional[set[str]]) – a memo to store the set of modules already added to the result

prefix (str) – a prefix that will be added to the name of the module

remove_duplicate (bool) – whether to remove the duplicated module instances in the

result or not

(str, Module) – Tuple of name and module

Duplicate modules are returned only once. In the following example, I will be returned only once.

Return an iterator over module parameters, yielding both the name of the parameter as well as the parameter itself.

prefix (str) – prefix to prepend to all parameter names.

recurse (bool) – if True, then yields parameters of this module and all submodules. Otherwise, yields only parameters that are direct members of this module.

remove_duplicate (bool, optional) – whether to remove the duplicated parameters in the result. Defaults to True.

(str, Parameter) – Tuple containing the name and parameter

Iterator[tuple[str, torch.nn.parameter.Parameter]]

Return an iterator over module parameters.

This is typically passed to an optimizer.

recurse (bool) – if True, then yields parameters of this module and all submodules. Otherwise, yields only parameters that are direct members of this module.

Parameter – module parameter

Register a backward hook on the module.

This function is deprecated in favor of `register_full_backward_hook()` and the behavior of this function will change in future versions.

a handle that can be used to remove the added hook by calling `handle.remove()`

`torch.utils.hooks.RemovableHandle`

Add a buffer to the module.

This is typically used to register a buffer that should not be considered a model parameter. For example, BatchNorm's `running_mean` is not a parameter, but is part of the module's state. Buffers, by default, are persistent and will be saved alongside parameters. This behavior can be changed by setting `persistent` to `False`. The only difference between a persistent buffer and a non-persistent buffer is that the latter will not be a part of this module's `state_dict`.

Buffers can be accessed as attributes using given names.

`name (str)` – name of the buffer. The buffer can be accessed from this module using the given name

`tensor (Tensor or None)` – buffer to be registered. If `None`, then operations that run on buffers, such as `cuda`, are ignored. If `None`, the buffer is not included in the module's `state_dict`.

`persistent (bool)` – whether the buffer is part of this module's `state_dict`.

Register a forward hook on the module.

The hook will be called every time after `forward()` has computed an output.

If `with_kwargs` is `False` or not specified, the input contains only the positional arguments given to the module. Keyword arguments won't be passed to the hooks and

only to the forward. The hook c